Actividad 3. Taller propuesto
Reconocimiento de caracteres manuscritos japoneses con una red convolucional

Informe de ejecución

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiante: Sergio Santiago Duarte Rojas
Código: 20261595002

Fecha: 10 de octubre de 2026

Universidad Distrital Francisco José de Caldas

Índice

1. Introducción **2**

2. Marco teórico (resumen) **2**
2.1. Convolución, pooling y la capa final 2
2.2. Normalización por lotes y dropout 2
2.3. Aumento de datos 2

3. Metodología **3**
3.1. Conjunto de datos 3
3.2. Arquitectura propuesta 3
3.3. Entrenamiento, evaluación y aumento 4

4. Resultados **4**
4.1. Exploración del conjunto 4
4.2. Entrenamiento 4
4.3. Evaluación en prueba 5
4.4. Aumento de datos 6

5. Análisis **7**
5.1. Por qué cada componente de la red 7
5.2. La brecha entre validación y prueba 7
5.3. Por qué se confunden su y tsu 8
5.4. Qué hizo y qué no hizo el aumento de datos 8

6. Conclusiones **8**

7. Referencias **9**

A. Código de los bloques completados **9**

1. Introducción

El cuaderno resuelto del docente clasifica los dígitos de MNIST con una red convolucional ya diseñada. El cuaderno propuesto pide repetir el flujo con otro conjunto de datos y completar cinco bloques `TODO`, explorar el conjunto, diseñar una red propia con ciertas restricciones, escribir el bucle de entrenamiento, evaluar en prueba y explicar el error más frecuente, e implementar un aumento de datos y medir su efecto. Se eligió KMNIST, caracteres manuscritos japoneses en escritura cursiva antigua, con el mismo formato que MNIST, imágenes de 28 por 28 en escala de grises y diez clases, pero con formas mucho más variables dentro de cada clase.

El cuaderno se ejecutó completo con PyTorch 2.12 en GPU, semilla 42 y 5 épocas. KMNIST se descargó real, sin el conjunto de respaldo que el cuaderno usa cuando falla la descarga, y las diez verificaciones automáticas dieron `[OK]`. La celda opcional que exporta las imágenes a carpetas para MATLAB no se ejecutó porque el trabajo se hizo en Python. Se agregaron dos celdas de análisis, una que muestra imágenes del par de clases más confundido y otra que repite la comparación del aumento de datos con tres semillas, como recomienda el aviso del propio cuaderno.

2. Marco teórico (resumen)

2.1 Convolución, pooling y la capa final

Una capa convolucional desliza filtros de 3 por 3 sobre la imagen y produce un mapa de activación por filtro. Con relleno de un píxel el mapa conserva el tamaño de la entrada, 28 por 28, y es el *max pooling* de 2 por 2 el que reduce a 14 y luego a 7 píxeles por lado, quedándose con el valor más alto de cada ventana. Después de dos bloques quedan 64 mapas de 7 por 7, 3 136 características, que una capa lineal convierte en diez *logits*. La softmax y la entropía cruzada van dentro de la pérdida, como en el cuaderno resuelto.

2.2 Normalización por lotes y dropout

La normalización por lotes (*batch normalization*) reescala las activaciones de cada canal para que en cada lote tengan media cero y varianza uno, y después aplica una escala y un desplazamiento que la red aprende, dos parámetros por canal. Con eso las activaciones no se desplazan de escala a medida que cambian los pesos de las capas anteriores, y el entrenamiento tolera tasas de aprendizaje mayores desde la primera época. El *dropout* apaga al azar una fracción de las características durante el entrenamiento, de modo que la capa siguiente no puede depender de unas pocas, y se desactiva en la evaluación.

2.3 Aumento de datos

El aumento de datos crea variantes de las imágenes de entrenamiento, aquí desplazamientos de hasta dos píxeles en cada eje, para que la red vea en cada época una versión distinta de cada imagen. Actúa como regularizador, hace el entrenamiento más difícil y, si el conjunto de prueba contiene las mismas variaciones, mejora la generalización. La forma de comprobarlo es comparar con y sin aumento con la misma semilla y el mismo número de épocas, y repetir con varias semillas antes de concluir.

3. Metodología

3.1 Conjunto de datos

KMNIST tiene 60 000 imágenes de entrenamiento y 10 000 de prueba, con una clase por cada uno de diez caracteres del silabario *hiragana* en su escritura antigua, *o*, *ki*, *su*, *tsu*, *na*, *ha*, *ma*, *ya*, *re* y *wo*. Cada clase agrupa varias formas de escribir el mismo carácter, porque antes de la reforma de 1900 un sonido podía escribirse con distintos caracteres de origen, y eso se ve en la Figura 1. El cuaderno aparta 5 000 imágenes como validación, entrena con 55 000 y normaliza con la media (0.1918) y la desviación (0.3484) del entrenamiento.

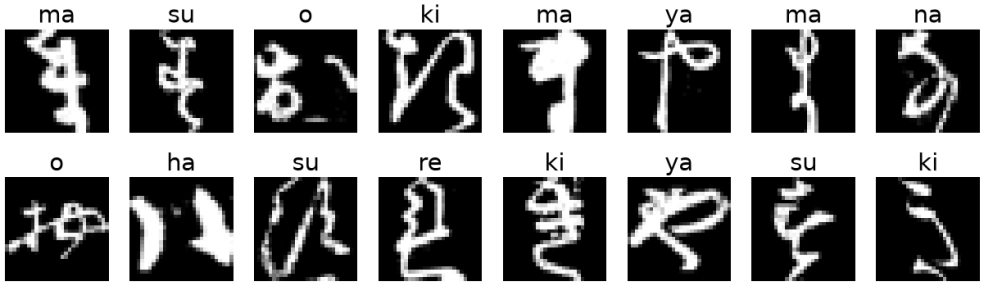


Figura 1: Primeras dieciséis imágenes de entrenamiento de KMNIST con su clase

3.2 Arquitectura propuesta

El enunciado pide al menos dos bloques de convolución con normalización por lotes y *pooling*, *dropout* antes de la capa final y menos de 200 000 parámetros. La red de la Tabla 1 y la Figura 2 cumple con 50 378 parámetros, una cuarta parte del límite. Cada componente tiene una razón. La convolución con relleno conserva el tamaño para que sea el *pooling* quien decida la reducción. Los 32 y 64 filtros duplican los 16 y 32 de la red del cuaderno resuelto porque las formas de KMNIST son más variadas que las de los dígitos. La normalización por lotes va justo después de cada convolución y antes de la ReLU, el orden habitual, para estabilizar el entrenamiento con la tasa de 10^{-3}. El *dropout* de 0.3 protege la capa lineal, que concentra el 62 por ciento de los parámetros (31 370) y es donde más riesgo hay de memorizar.

Tabla 1: Capas de la red propuesta. Las convoluciones usan relleno de un píxel y conservan el tamaño

Bloque	Capa	Parámetros
1	Convolución 3 por 3, 32 filtros, salida 32 por 28 por 28	320
	Normalización por lotes, ReLU, max pooling 2 por 2, salida 32 por 14 por 14	64
2	Convolución 3 por 3, 64 filtros, salida 64 por 14 por 14	18 496
	Normalización por lotes, ReLU, max pooling 2 por 2, salida 64 por 7 por 7	128
Salida	Aplanar, 3 136, y dropout 0.3	0
	Lineal 3 136 a 10	31 370
	Total	50 378

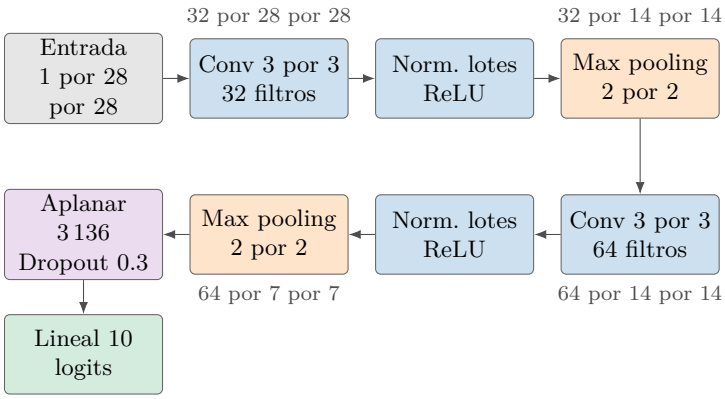


Figura 2: Red propuesta, 50 378 parámetros. Junto a cada bloque va el tamaño de su salida

3.3 Entrenamiento, evaluación y aumento

El bucle de entrenamiento sigue los cinco pasos de cada iteración, borrar los gradientes, propagar el lote, calcular la entropía cruzada, retropropagar y actualizar con Adam a tasa 10^{-3}, con lotes de 64 imágenes y 5 épocas. La prueba se usa una sola vez. El aumento desplaza cada lote un número entero aleatorio de píxeles entre -2 y 2 en cada eje con `torch.roll`, lo que sale por un borde entra por el opuesto. La comparación con y sin aumento se hace con la misma semilla y las mismas épocas, y se repite con las semillas 42, 43 y 44 para separar el efecto del ruido.

4. Resultados

4.1 Exploración del conjunto

Las diez clases tienen entre 5 450 y 5 533 imágenes de entrenamiento, una diferencia menor al 2 por ciento, así que el conjunto está balanceado y la exactitud global es una medida fiable. No hace falta ponderar las clases ni remuestrear.

4.2 Entrenamiento

La Tabla 2 y la Figura 3 muestran las 5 épocas. La pérdida de entrenamiento baja de 0.3055 a 0.0809 y la exactitud de validación sube de 95.96 a 97.62 por ciento, con su máximo de 97.68 en la época 4. Entrenamiento y validación terminan juntos, 97.60 y 97.62 por ciento, sin brecha de sobreajuste.

Tabla 2: Pérdida y exactitud por época de la red propuesta. La exactitud está en porcentaje

	Entrenamiento		Validación	
Época	**Pérdida**	**Exactitud**	**Pérdida**	**Exactitud**
1	0.3055	90.68	0.1378	95.96
2	0.1541	95.31	0.1027	96.80
3	0.1173	96.37	0.0890	97.56
4	0.0937	97.07	0.0794	97.68
5	0.0809	97.60	0.0808	97.62

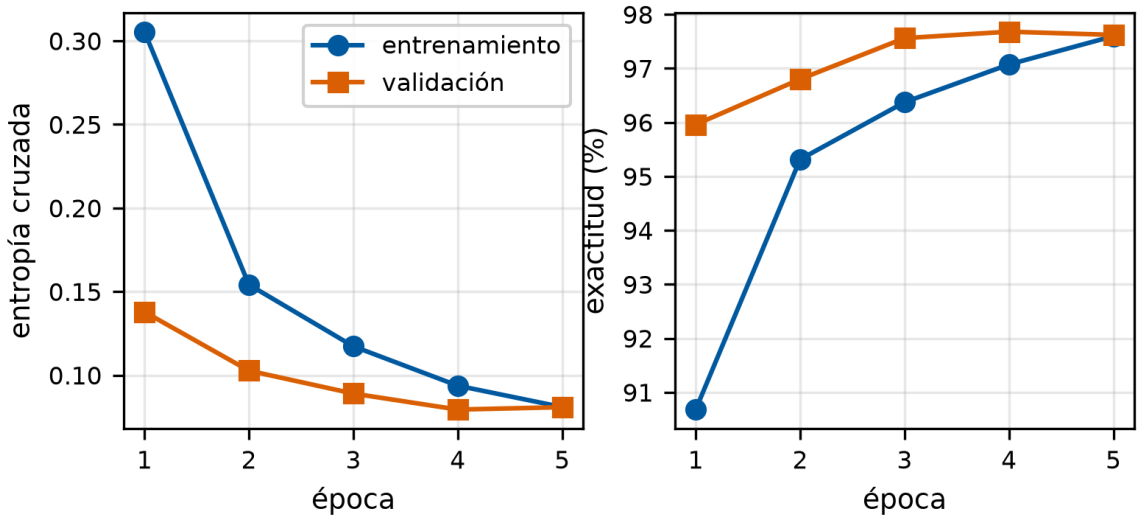


Figura 3: Pérdida y exactitud por época de la red propuesta en KMNIST

4.3 Evaluación en prueba

La exactitud en prueba es 93.34 por ciento, 666 errores de 10 000, con pérdida 0.2569, 4.3 puntos por debajo de la validación. La Figura 4 muestra la matriz de confusión y la Tabla 3 el acierto por clase, que va de 84.0 por ciento en su a 97.2 en re. El par más confundido es su tomado por tsu, 49 de las 1 000 imágenes de su. La columna na recibe 175 errores de casi todas las clases, el 26 por ciento de todos los errores. La Figura 5 muestra cinco imágenes del par confundido junto a imágenes bien clasificadas de las dos clases.

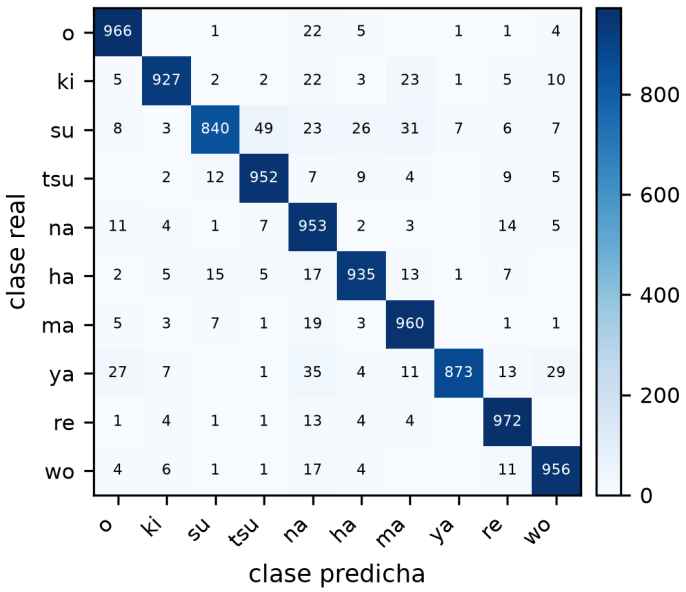


Figura 4: Matriz de confusión sobre las 10 000 imágenes de prueba

Tabla 3: Acierto por clase en prueba. El acierto es la fracción de imágenes de la clase bien clasificadas y la precisión la fracción de predicciones de esa clase que eran correctas

Clase	Imágenes	Aciertos	Acierto (%)	Precisión (%)
o	1000	966	96.6	93.9
ki	1000	927	92.7	96.5
su	1000	840	84.0	95.5
tsu	1000	952	95.2	93.4
na	1000	953	95.3	84.5
ha	1000	935	93.5	94.0
ma	1000	960	96.0	91.5
ya	1000	873	87.3	98.9
re	1000	972	97.2	93.6
wo	1000	956	95.6	94.0

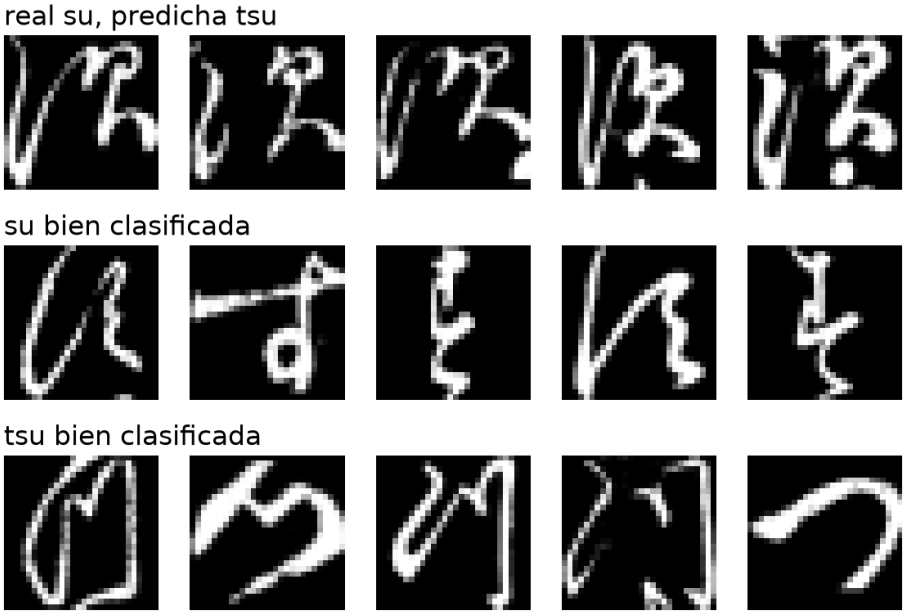


Figura 5: Arriba, imágenes de su que la red tomó por tsu. En medio y abajo, imágenes de su y de tsu bien clasificadas

4.4 Aumento de datos

La Figura 6 muestra el efecto del desplazamiento sobre cuatro imágenes. La Tabla 4 y la Figura 7 resumen tres semillas con 5 épocas cada una. Con aumento la exactitud de entrenamiento baja -2.3 puntos, la de validación cambia 0.0 y la de prueba -0.3, las dos últimas dentro de la desviación entre semillas. La brecha entre validación y entrenamiento pasa de 0.2 a 2.5 puntos.

Tabla 4: Exactitud en porcentaje, media y desviación de las semillas 42, 43 y 44 con 5 épocas

	Entrenamiento	Validación	Prueba
Sin aumento	97.50 ± 0.01	97.67 ± 0.22	93.26 ± 0.45
Con aumento	95.20 ± 0.15	97.68 ± 0.30	92.99 ± 0.38

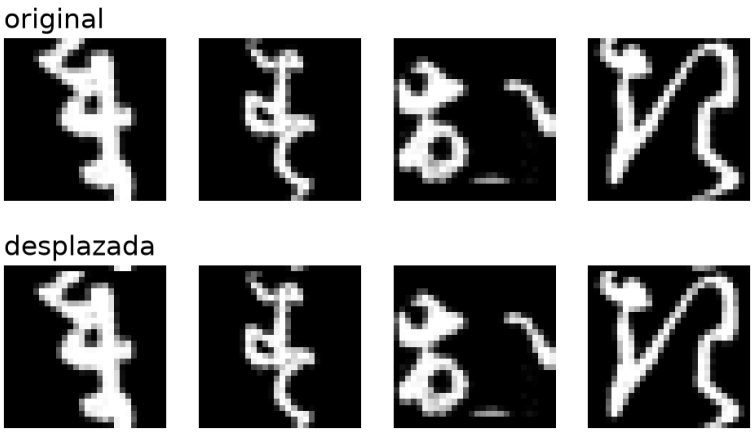


Figura 6: Cuatro imágenes de entrenamiento antes y después del desplazamiento aleatorio

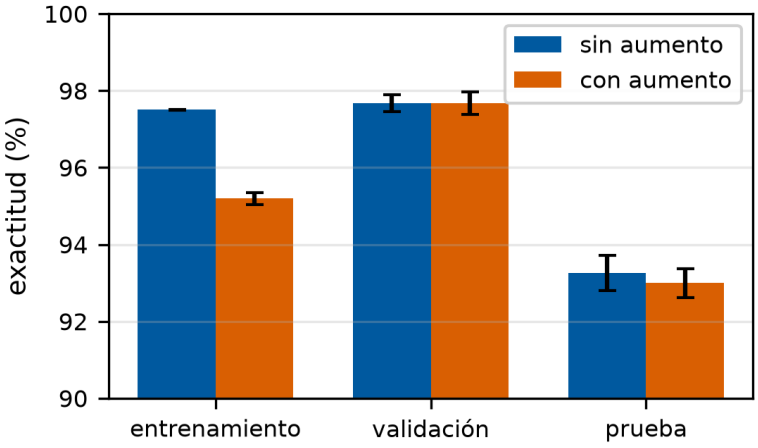


Figura 7: Exactitud media de las tres semillas con y sin aumento. Las barras de error son la desviación entre semillas

5. Análisis

5.1 Por qué cada componente de la red

La pregunta del paso 2 era qué se esperaba de cada componente. La convolución aporta filtros que se reutilizan en toda la imagen, y con 32 y 64 de ellos la red tiene margen para las variantes de forma de KMNIST. La normalización por lotes hizo lo que se esperaba, la red llegó a 95.96 por ciento de validación en la primera época con la tasa de 10^{-3}, sin ningún ajuste de la tasa. El *pooling* redujo las 3 136 características de la capa lineal a una cantidad manejable, y aun así esa capa concentra el 62 por ciento de los parámetros, lo que justifica el *dropout* delante de ella. La Tabla 2 muestra que la combinación funcionó, entrenamiento y validación avanzan juntos y la pérdida de validación baja en cada época.

5.2 La brecha entre validación y prueba

La validación termina en 97.62 por ciento y la prueba en 93.34, 4.3 puntos menos. No es sobreajuste, porque el entrenamiento y la validación coinciden y el *dropout* está activo. La validación es una muestra al azar del mismo conjunto de entrenamiento, escrita por las mismas manos y con las

mismas formas, mientras que la prueba de KMNIST proviene de otros documentos. Una validación tomada del entrenamiento no puede anticipar esa diferencia, y por eso la prueba se reserva para una sola medición final. En MNIST la brecha fue de dos décimas de punto porque sus dígitos de prueba se parecen mucho a los de entrenamiento. Esta diferencia entre los dos conjuntos es la lección más clara del taller.

5.3 Por qué se confunden su y tsu

El par más frecuente es su tomado por tsu. La Figura 5 lo explica. La clase su reúne varias formas antiguas del carácter, y las cinco imágenes mal clasificadas comparten dos trazos verticales largos con un lazo, mientras que las de su bien clasificadas muestran formas más compactas y distintas entre sí. Las imágenes de tsu bien clasificadas tienen también trazos verticales largos que se cierran, así que la variante alargada de su cae en el territorio visual de tsu. La red no falla por un defecto del entrenamiento sino porque una misma clase contiene formas que se parecen más a otra clase que entre sí. La columna na con el 26 por ciento de los errores muestra otra cosa, la red usa na como respuesta por defecto ante formas que no reconoce bien, probablemente porque sus trazos curvos y sueltos se parecen a fragmentos de muchos otros caracteres.

5.4 Qué hizo y qué no hizo el aumento de datos

El aumento no mejoró la exactitud de prueba ni la de validación. Las diferencias de 0.0 y -0.3 puntos están dentro de la desviación entre semillas, así que no se puede afirmar un cambio en ningún sentido. Lo que sí cambió es la brecha, que pasó de 0.2 a 2.5 puntos porque el entrenamiento cayó -2.3 puntos. Es el comportamiento esperado de un regularizador, la red ve imágenes más difíciles de las que después evalúa. Pero un regularizador solo ayuda cuando hay sobreajuste que corregir, y en 5 épocas no lo había. Hay además dos razones para que no ayude en la prueba. El *pooling* ya tolera desplazamientos de uno o dos píxeles, de modo que la red no necesitaba verlos durante el entrenamiento. Y los errores de prueba no vienen de la posición del carácter sino de su forma, como muestra el par su y tsu, y un desplazamiento no cambia la forma. Si se quisiera cerrar la brecha de prueba haría falta un aumento que cambie la forma, por ejemplo pequeñas rotaciones o deformaciones del trazo, y más épocas para que el regularizador tenga algo que corregir.

6. Conclusiones

La red propuesta de dos bloques con normalización por lotes y *dropout* alcanza 97.62 por ciento de validación y 93.34 de prueba en KMNIST con 50 378 parámetros, y cumple todas las restricciones del enunciado con las diez verificaciones en `[OK]`.

La brecha de 4.3 puntos entre validación y prueba no es sobreajuste. Las curvas de entrenamiento y validación coinciden, y la diferencia viene de que la prueba de KMNIST proviene de otros documentos. Una validación tomada del entrenamiento no puede detectar eso.

El error dominante, su tomado por tsu en 49 de 1 000 imágenes, nace de una clase que agrupa formas de escritura muy distintas, y la columna na concentra el 26 por ciento de los errores como respuesta por defecto de la red.

El aumento por desplazamiento de dos píxeles no cambió la exactitud de validación ni de prueba con tres semillas, y solo redujo la de entrenamiento. En 5 épocas no había sobreajuste que corregir, el *pooling* ya tolera esos desplazamientos y los errores son de forma y no de posición.

Repetir el experimento con tres semillas fue necesario. Con una sola, la primera comparación daba al aumento una ventaja de medio punto en prueba que desapareció al promediar.

7. Referencias

- Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., & Ha, D. (2018). Deep learning for classical Japanese literature. *NeurIPS Workshop on Machine Learning for Creativity and Design*.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning*.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

A. Código de los bloques completados

Se reproducen los cinco bloques `TODO` tal como quedaron en el cuaderno entregado, entre las marcas que usa el docente.

Código 1: TODO 1, imágenes por clase

```
conteo = torch.bincount(datos["y_tr"], minlength=len(clases))
```

Código 2: TODO 2, arquitectura

```
class CNNPropia(nn.Module):
    def __init__(self, p_drop=0.3):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)     # 28 -> 28
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)    # 14 -> 14
        self.bn2 = nn.BatchNorm2d(64)
        self.drop = nn.Dropout(p_drop)
        self.fc = nn.Linear(64 * 7 * 7, 10)
    def forward(self, x):
        x = F.max_pool2d(F.relu(self.bn1(self.conv1(x))), 2)        # 28 -> 14
        x = F.max_pool2d(F.relu(self.bn2(self.conv2(x))), 2)        # 14 -> 7
        return self.fc(self.drop(x.flatten(1)))
```

Código 3: TODO 3, los cinco pasos de una iteración

```
opt.zero_grad()
salida = modelo(xb)
perdida = F.cross_entropy(salida, yb)
perdida.backward()
opt.step()
```

Código 4: TODO 4, par más confundido

```
fuera = cm.copy(); np.fill_diagonal(fuera, 0)
par = tuple(int(v) for v in np.unravel_index(np.argmax(fuera), fuera.shape))
```

Código 5: TODO 5, desplazamiento aleatorio del lote

```
def aumentar_desplazamiento(xb, max_px=2):
    dy, dx = (int(v) for v in torch.randint(-max_px, max_px + 1, (2,)))
    return torch.roll(xb, shifts=(dy, dx), dims=(2, 3))
```